

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

КУРСОВА РОБОТА
Пояснювальна записка

з дисципліни: «Об'єктно-орієнтоване програмування»

Тема роботи: «Генеалогічне дерево»

Виконав
ст. гр. ПЗПІ-25-1

Голік І.С

Керівник:
Проф.

Володимир БОНДАРЄВ

Комісія:
Проф.
Ст. викл.
Ст. викл.

Володимир БОНДАРЄВ
Юлія ЧЕРЕПАНОВА
Віталій ЛЯПОТА

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра	Програмної інженерії		
Рівень вищої освіти	перший(бакалаврський)		
Дисципліна	Об'єктно-орієнтоване програмування		
Спеціальність	121 Інженерія програмного забезпечення		
Курс	1	Група	ПЗПІ-25-1
		Семестр	2

ЗАВДАННЯ
на курсову роботу здобувачеві

Голіку Івану Сергійовичу

1. Тема роботи: Генеалогічне дерево.
2. Термін подання здобувачем роботи до захисту “30” - травня - 2026 р.
3. Вихідні дані для роботи: Паспортні дані членів деякого родового клану; посилання на дітей (або на батьків). Пошук всіх нащадків або всіх предків для вказаної особи. Ієрархічне відображення генеалогічного дерева обраної людини.
4. Перелік питань, що потрібно опрацювати в роботі : Вступ, опис вимог, проєктування програми, інструкція користувача, висновки.

КАЛЕНДАРНИЙ ПЛАН

Номер	Назва етапів роботи	Термін виконання
1	Видача теми, узгодження і затвердження теми (1 тижд.)	25.03.2026 - 01.04.2026 р.
2	Формулювання мети роботи (1 тижд.)	01.04.2026 – 08.04.2026 р.
3	Опис вимог до програми (1 тижд.)	08.04.2026 – 15.04.2026 р.
4	Проектування програми (1 тижд.)	15.04.2026 – 22.04.2026 р.
5	Кодування (2 тижд.)	22.04.2026 – 06.05.2026 р.
6	Підготовка роботи до захисту (1 тижд.)	13.05.2026 – 20.05.2026 р.
7	Захист	20.05.2026 – 30.05.2026 р.

Здобувач _____

Голік І.С.,

Керівник _____

Проф. Володимир БОНДАРЄВ.

«21» _____ лютого 2026 р.

РЕФЕРАТ

Пояснювальна записка до курсової роботи: 20 с., 3 рис., 5 джерел.

ГЕНЕАЛОГІЧНЕ ДЕРЕВО, ООП, ПЕРСОНА, .NET, C#, JSON, WINFORMS

Метою роботи є розробка програми «Генеалогічне дерево», яка дозволяє зберігати, переглядати та аналізувати паспортні дані членів родового клану разом із зв'язками між ними.

У результаті отримана настільна програма на C Sharp [1] / WinForms [2], що підтримує операції додавання, редагування та видалення персон; пошук усіх предків або нащадків обраної особи; ієрархічне графічне відображення дерева; збереження і завантаження бази даних у форматі JSON [3].

В процесі розробки використано: Microsoft Visual Studio 2022, Windows Forms, платформа .NET [4] 8.0, мова програмування C Sharp [1].

ЗМІСТ

Вступ	6
1 Приклади сценаріїв використання	7
1.1 Сценарії використання	7
1.2 Функції програми	9
2 Постановка задачі	14
2.1 Архітектура та структура проєкту	14
2.2 Модель даних	14
2.3 Основні класи та зв'язки між ними	15
2.4 Алгоритми обробки даних	16
2.5 Зберігання даних	17
2.6 Інструкція зі встановлення	18
Висновки	19
Перелік джерел посилання	20

ВСТУП

Генеалогія - наука, що вивчає родинні зв'язки між людьми, походження та родовід окремих осіб, сімей і родів. Ще з давніх часів людство передавало інформацію про своїх предків від покоління до покоління в усній традиції, а згодом - у вигляді письмових родоводів і картотек. Із поширенням цифрових технологій така інформація стала надаватися до систематизації та зберігання у програмних системах.

Сучасне програмне забезпечення для ведення генеалогічних даних повинне вирішувати низку взаємопов'язаних завдань: зберігати великий масив паспортних і біографічних даних про членів родини; моделювати складні ієрархічні зв'язки між ними (батьки - діти, покоління тощо); забезпечувати зручний пошук як окремих осіб, так і всього ланцюжка предків або нащадків; наочно відображати родовід у вигляді дерева.

Програма «Генеалогічне дерево», розроблена в межах даної курсової роботи, призначена для ведення електронної картотеки родового клану. Типовий користувач програми - людина, яка бажає систематизувати інформацію про свій рід: записати паспортні дані родичів, вказати зв'язки між батьками та дітьми, знайти всіх предків або нащадків конкретної особи, а також наочно побачити дерево родоводу на екрані.

Програма реалізована мовою C Sharp із використанням технології Windows Forms на платформі .NET 8.0[4]. Вибір даного стеку технологій обумовлений вимогами навчальної дисципліни «Об'єктно-орієнтоване програмування», а також тим, що WinForms[2] забезпечує нативний зовнішній вигляд, широкий набір стандартних елементів керування та просту модель подій

1 ПРИКЛАДИ СЦЕНАРІЇВ ВИКОРИСТАННЯ

1.1 Сценарії використання

На основі аналізу предметної галузі визначені такі сценарії використання програми:

- додавання нової особи до бази даних;
- редагування даних наявної особи;
- видалення особи з бази даних;
- перегляд детальних відомостей про особу;
- фільтрація списку осіб за прізвищем або ім'ям;
- пошук усіх предків обраної особи;
- пошук усіх нащадків обраної особи;
- ієрархічне відображення генеалогічного дерева обраної особи;
- збереження бази даних у JSON-файл;
- завантаження бази даних із JSON-файлу.

Нижче буде наведено детальний опис деяких сценаріїв. Сценарії впорядковані за частотою використання - від найбільш типових до допоміжних.

1.1.1 Сценарій «Додавання особи»

Передумова: користувач відкрив головне вікно програми. Основний сценарій:

- 1) Користувач натискає кнопку «Додати».
- 2) Програма відкриває вікно додавання особи у модальному режимі.
- 3) Користувач заповнює поля (прізвище, ім'я обов'язкові; решта - за бажанням) і натискає «ОК».

4) Програма перевіряє коректність даних, створює запис і закриває вікно.

5) Нова особа з'являється у списку головного вікна.

Додатковий сценарій:

- 6) Користувач натискає «ОК» із незаповненими обов'язковими полями.
- 7) Програма виводить повідомлення про помилку, вікно залишається відкритим.

1.1.2 Сценарій «Перегляд генеалогічного дерева»

Передумова: користувач обрав особу зі списку.

Основний сценарій:

- 1) Програма автоматично будує дерево обраної особи.
- 2) Вгорі розміщуються предки (до 3 поколінь), обрана особа - по центру, нащадки - внизу (до 2 поколінь).
- 3) Вузли з'єднані лініями, обрана особа виділена кольором.

1.1.3 Сценарій «Редагування особи»

Передумова: користувач обрав особу зі списку. Основний сценарій:

- 1) Користувач натискає кнопку «Редагувати».
- 2) Програма відкриває вікно редагування із заздалегідь заповненими полями поточними даними.
- 3) Користувач змінює потрібні поля і натискає «ОК».
- 4) Програма валідує дані, оновлює запис і закриває вікно.
- 5) Список і деталі оновлюються автоматично.

1.1.4 Сценарій «Видалення особи»

Передумова: користувач обрав особу зі списку.

Основний сценарій:

- 1) Користувач натискає кнопку «Видалити» (або клавішу Delete).
- 2) Програма виводить діалог підтвердження з ім'ям особи.
- 3) Якщо особа є батьком або матір'ю інших осіб у базі, у діалозі з'являється відповідне попередження.
- 4) Після підтвердження запис видаляється, посилання на нього в інших записах обнуляються
- 5) Список та панель деталей очищуються.

Додатковий сценарій (відмова):

- 1) Користувач натискає «Ні» у діалозі підтвердження — запис залишається незмінним.

1.1.5 Сценарій «Пошук предків»»

Передумова: користувач обрав особу зі списку. Основний сценарій:

- 1) Користувач натискає кнопку «Усі предки».
- 2) Програма виконує рекурсивний обхід у ширину (BFS) від обраної особи вгору по ланцюжку батьків і дідів.
- 3) Результат відображається у діалоговому вікні у форматі «Прізвище Ім'я По-батькові (рік народження)».

Додатковий сценарій (предків не знайдено):

- 1) Якщо у обраної особи не зазначені батьки, програма виводить повідомлення «Не знайдено жодної особи».

1.2 Функції програми

Зі сценаріїв використання виведені такі функції програми. Нижче кожна функція описана з достатнім рівнем деталізації для реалізації та тестування.

1.2.1 Функція «Головне вікно»

Головне вікно програми поділене на три зони(див. рис 1.1):

- Ліва панель (270 пікселів): поле пошуку, список осіб, кнопки управління.
- Права верхня панель: деталі обраної особи (висота 200 пікселів).
- Права нижня панель: ієрархічне дерево (займає залишок простору).

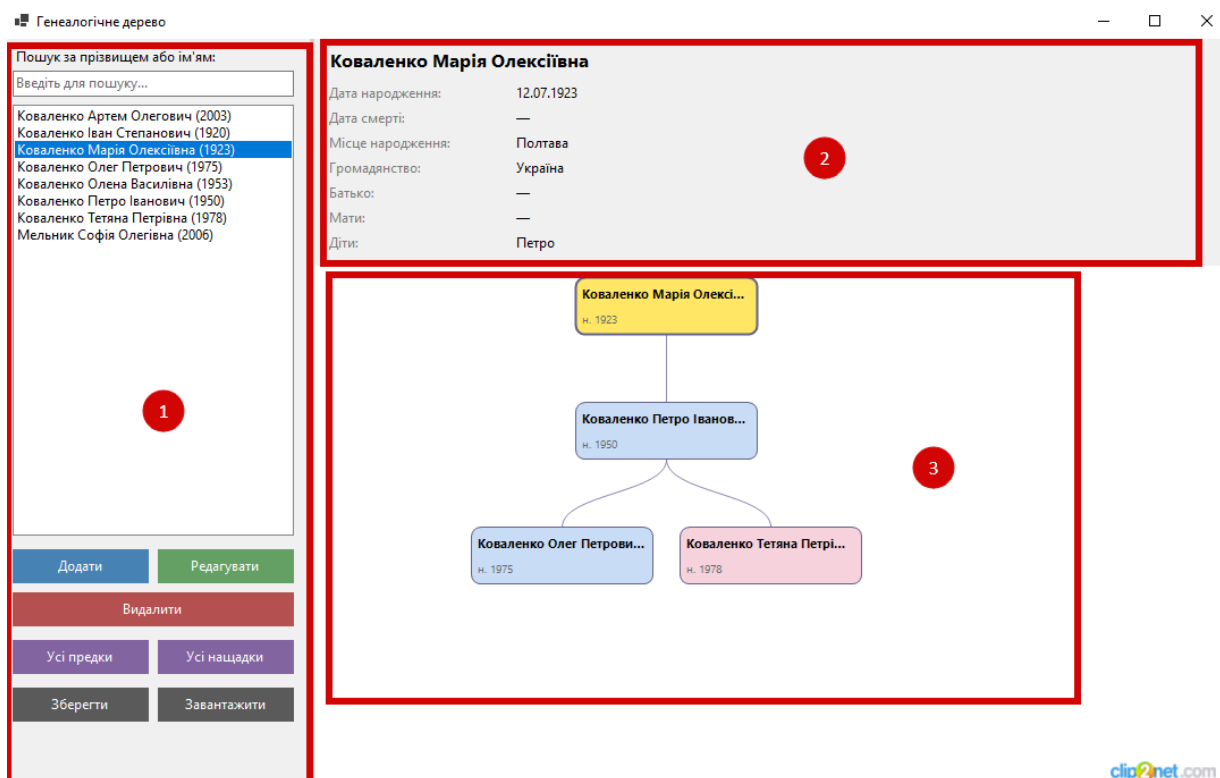


Рисунок 1.1 – Головне вікно програми

Список осіб відображає записи у форматі «Прізвище Ім'я По-батькові (рік)», відсортовані за прізвищем та ім'ям. При виборі запису оновлюються деталі та дерево.

Клавіатурні горячі клавіші: F1 - довідка; Delete (при фокусі на списку) - видалення. Введення тексту в поле пошуку миттєво фільтрує список без урахування регістру.

1.2.2 Функція «Панель деталей»

Функція є реалізацією сценарію «Перегляд списку осіб та деталей». При виборі особи зі списку у правій верхній панелі головного вікна відображаються детальні відомості про неї (рис. 1.1). Панель містить такі поля:

«Повне ім'я» - прізвище, ім'я та по батькові особи, виведені жирним шрифтом у верхній частині панелі;

«Дата народження» - дата у форматі дд.мм.рррр; якщо не вказана = відображається «—»;

«Дата смерті» - дата у форматі дд.мм.rrrr; якщо особа жива або дата невідома = відображається «—»;

«Місце народження» - рядок із назвою населеного пункту або «—»;

«Громадянство» - рядок або «—»;

«Батько» - повне ім'я батька, якщо він присутній у базі, або «—»;

«Мати» - повне ім'я матері, якщо вона присутня у базі, або «—»;

«Діти» - імена прямих нащадків через кому або «—», якщо дітей не зазначено;

«Примітки» - довільний текст або «—».

Коваленко Тетяна Петрівна	
Дата народження:	30.08.1978
Дата смерті:	—
Місце народження:	Харків
Громадянство:	Україна
Батько:	Коваленко Петро Іванович
Мати:	Коваленко Олена Василівна
Діти:	—
Примітки:	—

Рисунок 1.2 – Панель деталей персони

1.2.3 Функція «Додавання та редагування особи»

Вікно відкривається у модальному режимі після натискання кнопки «Додати» або «Редагувати» у головному вікні програми (рис. 1.2). При редагуванні всі поля заздалегідь заповнені поточними даними обраної особи. Вікно містить такі поля введення:

«Прізвище» - обов'язкове поле, рядок від 1 до 200 символів; позначене зірочкою;

«Ім'я» - обов'язкове поле, рядок від 1 до 200 символів; позначене зірочкою;

«По батькові» - необов'язкове текстове поле;

«Дата народження» - вибір дати за допомогою стандартного елемента DateTimePicker; якщо дата невідома - встановлюється позначка «невідома», поле вибору дати блокується;

«Дата смерті» - вибір дати; якщо особа жива - встановлюється позначка «жива особа», поле вибору дати блокується;

«Місце народження» - необов'язкове текстове поле;

«Громадянство» - необов'язкове текстове поле, за замовчуванням «Україна»;

«Стать» - випадаючий список із значеннями: Чоловіча, Жіноча, Невідома;

«Батько» - випадаючий список наявних у базі осіб чоловічої статі; перший елемент «— не вказано —»;

«Мати» - випадаючий список наявних у базі осіб жіночої або невідомої статі; перший елемент «— не вказано —»;

«Примітки» - багаторядкове текстове поле для довільних нотаток.

Редагування особи

Прізвище *	
Ім'я *	Тетяна
По батькові	Петрівна
Дата народження	30/08/1978
	☐ невідома
Дата смерті	10/05/2026
	☒ жива особа
Місце народження	Харків
Громадянство	Україна
Стать	Жіноча
Батько	Коваленко Петро Іванович (1950)
Мати	Коваленко Олена Василівна (1953)
Примітки	
OK Скасувати	

Рисунок 1.3 – Панель деталей персони

1.2.4 Функція «Видалення особи»

Після натискання «Видалити» програма виводить модальний діалог підтвердження. Текст діалогу містить повне ім'я особи. Якщо особа є зазначеним

батьком або матір'ю принаймні одного запису в базі, до тексту додається попередження. Після підтвердження:

- запис особи видаляється з колекції;
- у всіх інших записах поле FatherId або MotherId, що посилалося на видалену особу, встановлюється в null.

1.2.5 Функція «Пошук предків та нащадків»

Алгоритм реалізований методом обходу в ширину (Breadth-First Search, BFS). Для пошуку предків черга ініціалізується ідентифікатором обраної особи. На кожній ітерації:

- 1) Беруться значення FatherId та MotherId поточної особи.
- 2) Якщо ідентифікатор не порожній і відповідний запис не був відвіданий — він додається до результату і до черги.
- 3) Обхід продовжується до вичерпання черги.

1.2.6 Функція «Відображення генеалогічного дерева»

При виборі особи зі списку права нижня панель автоматично перебудовується. Алгоритм відображення:

- 1) Обрана особа розміщується на центральному рівні.
- 2) Батьки і діди додаються на рівні -1 , -2 , -3 (вище за обрану особу, до 3 поколінь).
- 3) Діти й онуки додаються на рівні $+1$, $+2$ (нижче, до 2 поколінь).
- 4) Рівні нормалізуються: мінімальний рівень = 0.
- 5) Вузли кожного рівня рівномірно розподіляються по горизонталі.
- 6) Зв'язки між вузлами малюються кривими Безьє.

Зовнішнє оформлення вузлів:

- жовтий фон - обрана особа;
- блакитний фон - особи чоловічої статі;
- рожевий фон - особи жіночої статі;
- сірий фон - стать невідома.

2 ПОСТАНОВКА ЗАДАЧІ

2.1 Архітектура та структура проєкту

Програма є настільним додатком з графічним інтерфейсом користувача на базі технології Windows Forms (.NET 8.0 [4], C Sharp [1]). Вибір архітектури обумовлений вимогами до платформи: WinForms [2] забезпечує нативний зовнішній вигляд на Windows, повний контроль над малюванням елементів засобами GDI+, зручну подійну модель та просте розгортання без додаткових залежностей. Програма дотримується принципу розділення відповідальностей: логіка роботи з даними зосереджена в класі FamilyRepository, відображення дерева - в TreeRenderer, взаємодія з користувачем - у Form1 і PersonEditForm. Це спрощує тестування та подальшу підтримку коду. Проєкт складається з таких файлів. У папці Models розміщено файл Person.cs, що містить модель даних: клас Person, клас FamilyRepository та перелік Gender. У папці Forms знаходиться PersonEditForm.cs - модальна форма додавання та редагування особи. У папці Services розміщено TreeRenderer.cs - клас, що реалізує алгоритм побудови і малювання генеалогічного дерева на панелі. В корені проєкту розташовані Form1.cs та Form1.Designer.cs, що реалізують головну форму програми, а також Program.cs - точка входу, та GenealogyTree.csproj - файл проєкту MSBuild.

Вихідний код проєкту [5] має в собі всі використані під час розробки ресурси

2.2 Модель даних

Основною одиницею даних є клас Person, що містить усі паспортні та генеалогічні відомості одного члена родини. Кожна особа має унікальний цілочисельний ідентифікатор Id, що призначається автоматично при додаванні до колекції. Паспортні дані представлені полями LastName (прізвище), FirstName (ім'я) та Patronymic (по батькові). Дати народження і смерті зберігаються як nullable-значення типу DateTime: відсутність значення означає, що дата невідома або особа жива відповідно. Місце народження та громадянство зберігаються як рядки, що допускають значення null. Стать особи зберігається через окремий перелік Gender із значеннями Male, Female та Unknown, що забезпечує типобезпечне зберігання без

використання рядкових констант. Поле Notes призначене для довільних нотаток та особливих прикмет. Зв'язки між особами реалізовані через зовнішні ключі FatherId та MotherId - nullable цілі числа, що посиляються на Id відповідних записів у тій самій колекції. Обидва поля допускають значення null, що дозволяє додавати особу без відомих батьків. Завдяки такому підходу одна плоска колекція List Person підтримує довільно складні родинні зв'язки без потреби у реляційній базі даних. Клас Person також містить обчислювані властивості FullName та DisplayInfo. Вони динамічно формуються з базових полів і позначені атрибутом [JsonIgnore], тому не зберігаються у файл і не займають додаткового місця.

2.3 Основні класи та зв'язки між ними

Програма складається з кількох основних класів, кожен з яких має чітко визначену відповідальність. Клас Person є моделлю даних і зберігає всі відомості про одну особу. Перелік Gender забезпечує типобезпечне представлення статі. Клас FamilyRepository є центральним сервісом роботи з даними: він агрегує колекцію List Person і є єдиним місцем, де виконуються будь-які операції над нею - додавання, оновлення, видалення, пошук, а також серіалізація. Form1 реалізує головне вікно програми зі списком осіб, панеллю деталей та кнопками управління. PersonEditForm реалізує модальну форму додавання та редагування особи з валідацією введених даних. Клас TreeRenderer є сервісом відображення: він отримує Panel через конструктор і реалізує алгоритм побудови та малювання дерева засобами GDI+. TreeNode є внутрішнім допоміжним класом, що представляє вузол дерева з координатами для малювання. GraphicsExtensions є статичним допоміжним класом із методами розширення для System.Drawing.Graphics, що реалізує малювання прямокутників із заокругленими кутами через GraphicsPath.

Form1 та PersonEditForm отримують посилання на репозиторій через конструктор, що є спрощеним варіантом впровадження залежності. TreeRenderer не залежить від жодного класу форми, що дозволяє легко перенести або протестувати компонент відображення незалежно від інтерфейсу.

Основні публічні методи FamilyRepository такі. Метод Add приймає об'єкт Person, призначає йому унікальний Id і додає до колекції. Метод Update замінює існуючий запис із відповідним Id. Метод Delete видаляє запис за ідентифікатором

і одночасно обнуляє поля `FatherId` та `MotherId` у всіх інших записах, де вони посилалися на видалену особу - це гарантує цілісність даних. Метод `GetAll` повертає незмінний список усіх осіб. Метод `GetById` шукає особу за ідентифікатором. Метод `GetChildren` повертає список прямих нащадків обраної особи. Методи `GetDescendants` та `GetAncestors` реалізують рекурсивний пошук усіх нащадків і предків відповідно. Методи `SaveToFile` та `LoadFromFile` виконують серіалізацію та десеріалізацію бази даних.

2.4 Алгоритми обробки даних

2.4.1 Алгоритм пошуку предків і нащадків (BFS)

Пошук усіх предків реалізований методом `GetAncestors` класу `FamilyRepository` методом обходу в ширину (`Breadth-First Search`). Черга ініціалізується ідентифікатором обраної особи, а множина відвіданих вузлів - порожньою. Поки черга не вичерпана, з неї вилучається поточний ідентифікатор; якщо він вже відвіданий - ітерація пропускається. Інакше вузол позначається як відвіданий, отримується відповідний запис особи. Якщо поле `FatherId` не порожнє і батько ще не потрапив до результату - він додається до результату і до черги. Аналогічно обробляється поле `MotherId`. Алгоритм завершується, коли черга порожня, і повертає список знайдених осіб. Для нащадків логіка симетрична: замість батьків шукаються всі особи, у яких `FatherId` або `MotherId` збігається з поточним ідентифікатором. Множина відвіданих вузлів виключає нескінченний цикл навіть при циклічних посиланнях у даних. Складність алгоритму - $O(V + E)$, де V - кількість осіб у базі, E - кількість зв'язків між ними.

2.4.2 Алгоритм побудови дерева для відображення

При виборі особи зі списку клас `TreeRenderer` перебудовує внутрішнє представлення дерева. Спочатку для обраної особи створюється центральний вузол з рівнем 0. Далі рекурсивно додаються предки: кожен батьківський вузол отримує рівень на одиницю менше за рівень дочірнього, а до його списку `Children` додається посилання на вузол нащадка - так формуються лінії зв'язку, що йдуть вниз. Аналогічно рекурсивно додаються нащадки з рівнями на одиницю більшими.

Після побудови рівні нормалізуються: мінімальний рівень зсувається до нуля, щоб предки опинилися зверху. На етапі розміщення для кожного рівня вузли рівномірно розподіляються по горизонталі, центруючись відносно ширини полотна. Мінімальна координата X гарантується не меншою за 10 пікселів. Розміри полотна оновлюються поза обробником події Paint, щоб уникнути рекурсивного виклику перемалювання. Вузли малюються із заокругленими кутами, кольоровим фоном залежно від статі та підписами імені й року народження в один рядок без переносу. З'єднувальні лінії між вузлами малюються кривими Безьє.

Мигання усунуто двома механізмами: увімкненням подвійним буферуванням панелі (DoubleBuffered) через рефлексію та виключенням будь-яких змін стану поза методом OnPaint.

2.4.3 Валідація введених даних

У формі PersonEditForm перед збереженням запускається метод ValidateFields, що збирає список текстових описів помилок. Перевіряється, що поля прізвища та імені не є порожніми і не складаються лише з пробільних символів. Якщо вказані обидві дати, перевіряється, що дата смерті не передуює даті народження. Якщо список помилок непорожній, він відображається у Label із червоним кольором у верхній частині вікна, а вікно залишається відкритим. Перед збереженням кінцеві пробіли у всіх текстових полях обрізаються методом Trim.

2.5 Зберігання даних

Дані серіалізуються у формат JSON [3] за допомогою стандартної бібліотеки System.Text.Json, що входить до складу .NET 8.0 [4]. Файл містить масив об'єктів Person та значення лічильника NextId, що зберігає поточне значення автоінкременту ідентифікаторів. Це необхідно для того, щоб після завантаження файлу нові записи отримували унікальні ідентифікатори, не конфліктуючи з вже наявними. Серіалізація виконується з параметром WriteIndented = true, що робить файл читабельним і придатним для ручного редагування за потреби. Обчислювані властивості FullName та DisplayInfo не серіалізуються завдяки атрибуту [JsonIgnore]. Формат JSON файлу виглядає наступним чином:

```

{
  "Persons": [
    {
      "Id": 1,
      "LastName": "Коваленко",
      "FirstName": "Іван",
      "Patronymic": "Степанович",
      "BirthDate": "1920-03-05T00:00:00",
      "DeathDate": null,
      "BirthPlace": "Харків",
      "Citizenship": "Україна",
      "Gender": 0,
      "FatherId": null,
      "MotherId": null,
      "Notes": null
    }
  ],
  "NextId": 9
}

```

2.6 Інструкція зі встановлення

Для роботи програми необхідна операційна система Windows 10 або новіша (64-розрядна) та встановлений .NET 8.0[4] Desktop Runtime. Наявність середовища розробки не є обов'язковою для кінцевого користувача. Перед встановленням слід перевірити наявність .NET 8.0, виконавши у командному рядку команду `dotnet --version`. Якщо версія нижча за 8.0 або команда не знайдена, необхідно завантажити та встановити .NET 8.0 Desktop Runtime з офіційного сайту Microsoft за адресою <https://dotnet.microsoft.com/download/dotnet/8.0>. Для запуску програми потрібно розпакувати архів `GenealogyTree_CSharp_WinForms.zip` у будь-яку зручну папку. Після цього програму можна запустити одним із двох способів: відкрити файл `GenealogyTree.csproj` у Visual Studio 2022 і натиснути клавішу F5, або відкрити командний рядок у папці проєкту і виконати команду `dotnet run`. Для отримання автономного exe-файлу, що не потребує встановленого .NET Runtime на цільовому комп'ютері, необхідно виконати команду `dotnet publish -c Release -r win-x64 --self-contained -o ./publish`. Готовий файл `GenealogyTree.exe` буде розміщено у папці `publish`.

Для видалення програми достатньо видалити папку з файлами проєкту. Програма не вносить змін до реєстру Windows і не використовує системні папки.

ВИСНОВКИ

У ході виконання курсової роботи розроблено програму «Генеалогічне дерево» мовою C Sharp[1] з використанням технології Windows Forms на платформі .NET 8.0[4]. Програма призначена для ведення електронної картотеки членів родового клану та аналізу родинних зв'язків між ними. У процесі виконання роботи були пройдені всі етапи розробки програмного забезпечення: формулювання мети та вимог, проєктування архітектури і моделі даних, кодування з дотриманням C Sharp[1] coding conventions, тестування функціональності. Реалізовані такі функції програми:

- повний цикл CRUD-операцій (додавання, редагування, видалення, перегляд) над записами осіб із валідацією введених даних;
- фільтрація списку осіб за прізвищем та ім'ям у реальному часі;
- рекурсивний пошук усіх предків і нащадків обраної особи методом BFS (обхід у ширину);
- ієрархічне графічне відображення генеалогічного дерева з кольоровим розрізненням статі, кривими Безьє та підтримкою прокрутки;
- збереження та завантаження бази даних у форматі JSON [3].

Програма повністю відповідає поставленим вимогам. Стійкість забезпечена перехопленням винятків у всіх операціях з файлами та валідацією форм перед збереженням. Цілісність даних при видаленні гарантується обнуленням зовнішніх ключів. Інтерфейс орієнтований на кінцевого користувача і не містить технічної термінології розробника. Можливі напрями вдосконалення проєкту:

- реалізація друку та експорту дерева у форматах PDF або PNG;
- додавання підтримки фотографій для кожної персони;
- розширення відображення дерева до необмеженої кількості поколінь із масштабуванням;

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. C# Programming Language. Anders Hejlsberg. URL: <https://dotnet.microsoft.com/en-us/languages/csharp> (дата звернення: 01.05.2026).
2. WinForms. .NET Team. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/> (дата звернення: 01.05.2026).
3. JSON (JavaScript Object Notation). Douglas Crockford. URL: <https://www.json.org/json-en.html> (дата звернення: 01.05.2026).
4. .NET 8.0. .NET Team. URL: <https://versionsof.net/core/8.0/8.0.20/> (дата звернення: 01.05.2026).
5. репо. Голік І. С.. URL: <https://github.com/johanmoses79/familytree> (дата звернення: 19.05.2026).

()